

Flow Free Solver + Screen Assistant

figuring out how to solve every Flow Free variant — and then how to put it on my phone, over the game.

o) The problem

- connect every pair of coloured dots, paths must not cross, and —
 - ↳ every single cell must be filled. (this is the killer rule)
 - ↳ that 3rd rule is why "just connect the dots" fails. It forces a **unique** answer.

o) Is it hard? **Yes**

- ↳ it's Numberlink → deciding solvability is NP-complete.
- ↳ no polynomial algo. Must use search + hard pruning, or reduce to SAT.
 - ↳ BUT real levels are **designed to be human-solvable** → they collapse fast once pruning kicks in.

o) Prior art I read

- Matt Zucker — fast search solver, most-constrained colour + connectivity pruning. (also did a SAT version)
- Torwaney — clean SAT reduction, K bool vars per **edge**. ← this is where the edge idea came from
- Columbia ParallelFlow paper.

My decision: search, not SAT. No solver dependency, single pass, ports to a phone, and I can explain each move.

o) KEY / LEGEND

read this once — every board in these notes uses it.

On the board

-  endpoint — the dot you see in the game
-  pipe — the path you draw
-  WALL — flow can't cross. both cells still exist & must be filled
-  HOLE — not a cell. must NOT be filled
-  BRIDGE — 2 flows cross here (1 H + 1 V)

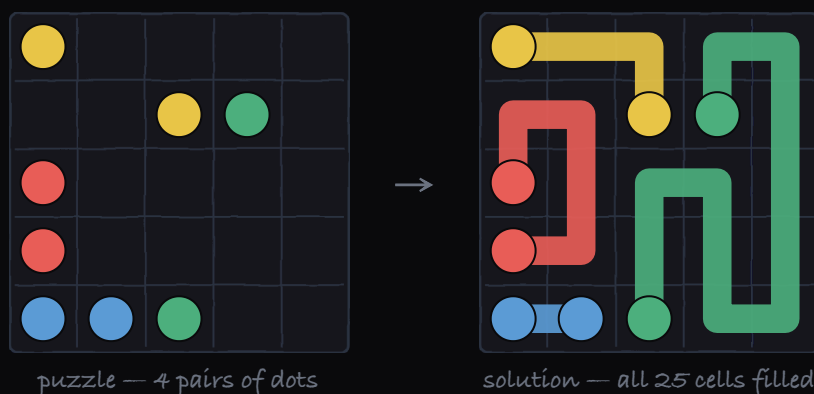
The 4 rules

1. each pair joined by one path
2. paths never cross
3. every cell filled ← the hard one
4. endpoints = degree 1
other cells = degree 2

Colours

-  flow A
-  flow B
-  flow C
-  flow D
-  flow E
-  flow F
-  flow G
-  flow H

o) Worked example — plain 5x5



↳ see how C takes the **long way round**? it has to — because every cell must be filled. That's rule 3 doing all the work.

It is ONE GRAPH.

Flow Free ships ~30 pack types — Mania, Bridges, Walls, Obstacles, Cubes, Rectangle, Links, Warps, Hexes...

↳ they look like different games. They are not.

↳ every one = the same puzzle on a different graph.

Board = (NODES , EDGES , BRIDGES)

NODES = playable cells · coverage = every NODE filled

EDGES = legal moves · a flow may ONLY step along an edge

o) Every variant = a transform, done at **parse time**

→ Walls remove EDGES (cells stay!)

→ Obstacles / holes .. remove NODES

→ Cubes / Warps / portals .. add EDGES

→ Rectangle, Mania nothing. $R \neq C$, or just bigger.

→ Bridges mark a node dual-pass ← **only one that touches the solver**

↳ remove edges. remove nodes. add edges. 3 operations = the entire game.

2 rules I must never break:

1. neighbours come from **edges** — **NEVER** $r \pm 1$ arithmetic. (kills walls, seams, portals — silently)

2. coverage counts **nodes** — **NEVER** rows × cols. (holes leak back in → "everything unsolvable")

↳ proved it: I built the solver, ran 7 variants. Only bridges touched the search loop.

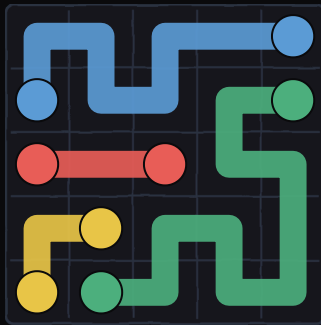
this is the whole project in one page. everything else is detail.

o) WALLS = remove an EDGE

a bold line between two cells. Flow cannot cross it.

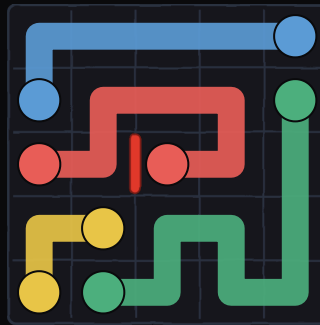
↳ both cells **still exist**. Both still have to be filled. You just go around.

Same endpoints — wall off vs wall on



no wall — A goes straight across

vs



+1 wall — A detours, everything reorganises

↳ look at A: endpoints 2 cells apart on the same row. Without the wall it goes straight across.

Add the wall → it must detour through the row above, and every other flow reorganises.

↳ so a wall fixture must be **load-bearing** — otherwise a solver that ignores walls still passes the test.

THE TRAP — wall ≠ obstacle. They look the same. They are opposites.

WALL

- ↳ both cells exist
- ↳ must be covered
- ↳ remove the **edge**

OBSTACLE

- ↳ the cell is gone
- ↳ must **NOT** be covered
- ↳ remove the **node**

get it backwards → solver chases a cell that can never be filled →

EVERY board reports unsolvable.

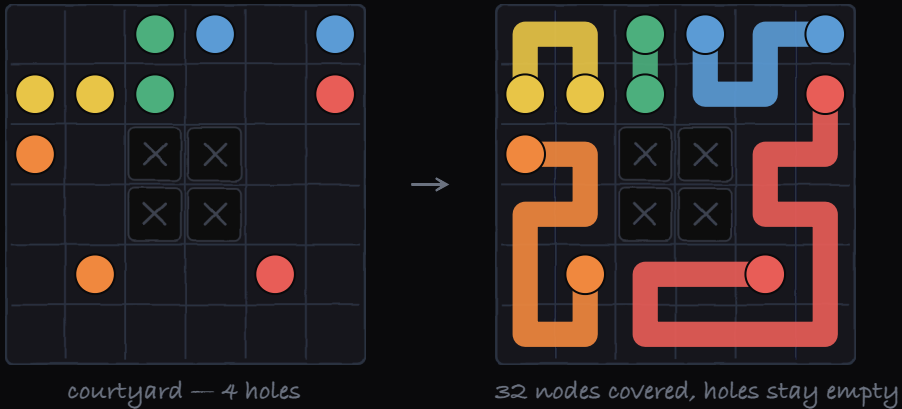
↳ I tested it: fenced one cell on all 4 sides instead of removing it. Solver said

UNSOLVABLE. Exactly as predicted.

○) OBSTACLES = remove a NODE

the cell is not part of the board. No flow enters it, and —

↳ it does NOT need to be filled.



courtyard — 4 holes

32 nodes covered, holes stay empty

coverage = "every NODE filled" — **not** "every cell in the $R \times C$ rectangle"

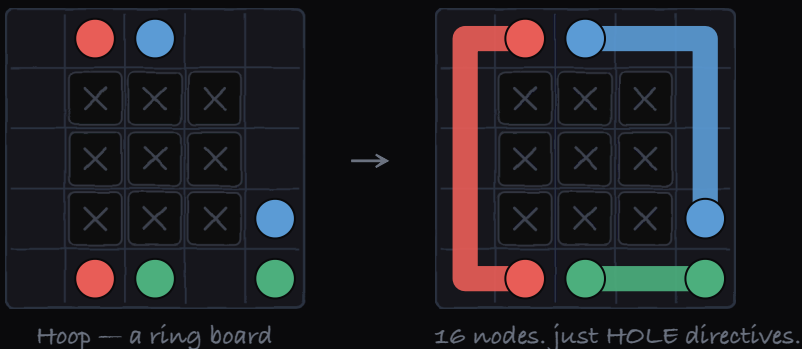
↳ if holes leak into the count, the solver waits forever for a cell that can never exist → every board unsolvable. Symptom points nowhere near the cause.

○) Shaped boards just compose

Worm, Amoeba, Inkblot, Courtyard, Hourglass, Hoop/Loop/Chain —

↳ all of them are **just HOLE directives**. **no new code**

↳ there is no "Amoeba solver". A ring board is a rectangle with the middle punched out.

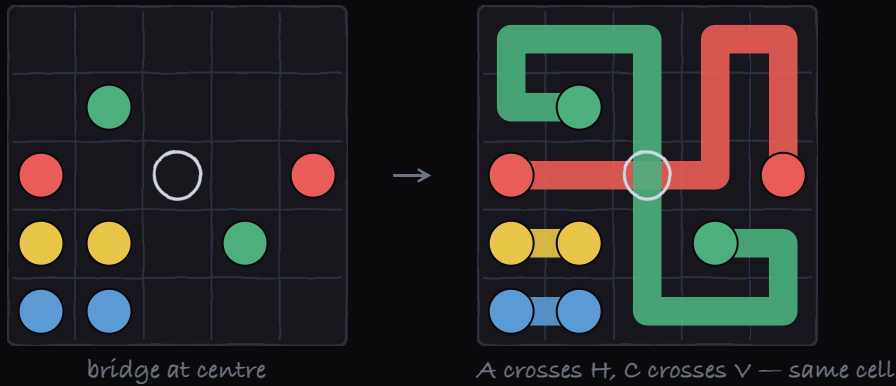


Hoop — a ring board

16 nodes, just HOLE directives.

o) BRIDGES = a dual-pass node

two flows cross the same cell without connecting. one goes **horizontal**, one goes **vertical**.



↳ rules: 2 passes (1 H + 1 V) · must be different colours · each goes **STRAIGHT** through (no turning on a bridge!)

↳ covered only when both passes are filled → it counts as **2 units** of coverage, not 1.

THIS is why colour lives on **EDGES**, not cells.

`owner[r][c]` can hold A or C — **not both**.

↳ a cell-based flood fill sees the bridge as "occupied by A" and refuses to route C through it → board looks unsolvable.

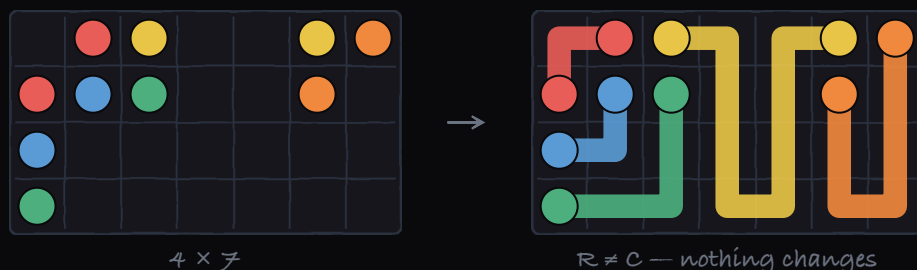
↳ so: build the edge model **FIRST**. Retrofitting = rewriting every flood fill.

↳ good news: a flow entering a bridge has its next move forced (straight out the far side).

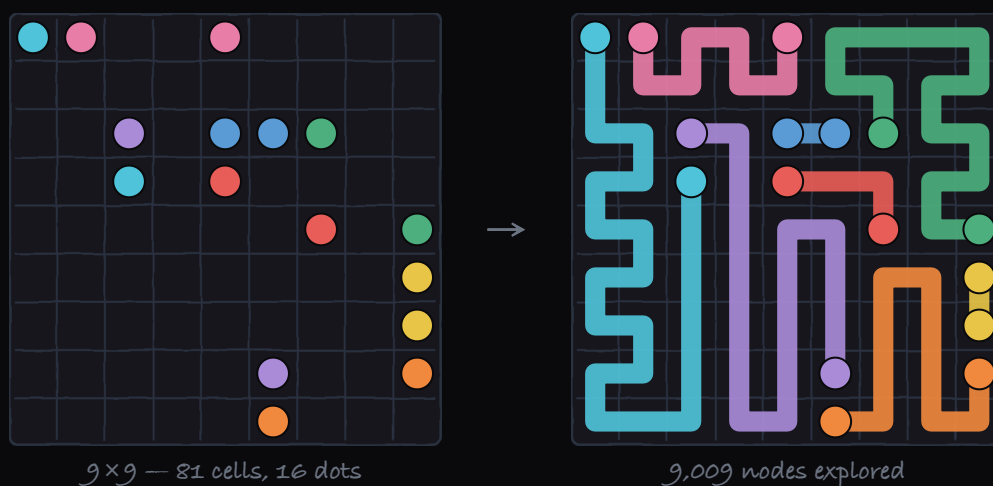
Bridges **shrink** the search.

o) RECTANGLE — Tower, Hourglass, Shifted

$R \neq C$. Nothing changes. But: never store a single `size` field or a detector that finds 9 rows will assume 9×9 and mis-parse everything.



o) MANIA — up to 15×15 . The scale test.



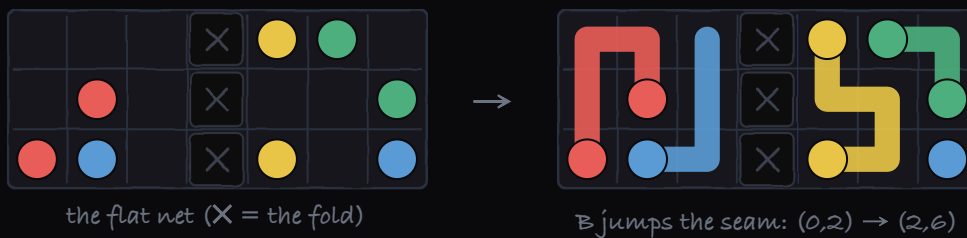
↳ 81 cells, only 16 endpoints. Coverage is what pins it down — G and F snake across half the board *because every cell must be filled*.

↳ search cost: 9,009 nodes vs ~ 24 for a 5×5 . ← that's the blow-up the pruning has to fight.

o) CUBES = add EDGES (seams)

board is a folded net. Flows continue across the fold onto the next face.

↳ cells on adjoining face edges are **adjacent in 3D** — even though they're far apart in the flat net.



↳ flow B steps from (0,2) → (2,6). In the flat net that looks like a teleport. On the folded cube they are simply neighbours.

The seam map must be EXPLICIT DATA. Not index arithmetic.

↳ folds meet with **rotations** — face R's left column may map to face F's right column reversed.

↳ get the reversal backwards → board still looks solvable, hands you a confident answer, and the game rejects it. There is no crash to debug.

↳ hardcode a table: (faceA, edgeA, faceB, edgeB, reversed?). Boring = correct at 3am.

↳ same "add an edge" trick covers **wraps** (wrap opposite borders) and **portals**. All one operation.

o) PARITY — walls & holes are different boards

colour the grid like a chessboard: $(r+c) \% 2$. The graph is bipartite $\rightarrow$ a flow alternates black/white every step.

$\rightarrow$ both endpoints same colour $\rightarrow$ odd length $\rightarrow$ eats one extra of that colour (± 1)

$\rightarrow$ endpoints opposite colours $\rightarrow$ equal number of each (0)

full coverage $\Rightarrow$ the flows partition every node. So sum it all up:

$$(\text{\#even} - \text{\#odd}) = S - T$$

S = colours with BOTH endpoints on even cells

T = colours with BOTH endpoints on odd cells

$\rightarrow S - T$ depends ONLY on where the endpoints are.

o) I measured it

board	nodes	#even - #odd	$S - T$
plain 5x5	25	+1	+1
5x5 + WALL	25	+1	+1
5x5 + HOLE	24	0	0
6x6 courtyard	32	0	0

$\rightarrow$ the walled board needs $S - T = +1$. The holed board needs $S - T = 0$.

The same endpoints can NEVER solve both. Removing a node always flips the parity budget by 1.

Free win: compute $S - T$ at parse time, $O(\text{cells})$, before any search.

$\rightarrow$ if it disagrees $\rightarrow$ board is unsolvable, don't search.

$\rightarrow$ catches a wall misread as a hole instantly — i.e. the exact bug my screen-reader is most likely to make.

$\rightarrow$ (doesn't apply with bridges — a bridge node is visited twice, so it's not a partition.)

found this by accident — tried to draw a diagram that turned out to be mathematically impossible.

o) THE ALGORITHM

`solve(state):`

if not `prune_ok(state)` → **FAIL** ← dead state, backtrack
`apply_forced_moves(state)` ← single-option moves, no branching
 if all done → **SUCCESS** iff `emptyCount == 0`
`c = most_constrained_colour()` ← fewest legal moves
 for each move of `c`: push → recurse → pop

o) The 5 prunes — this is where ALL the speed is

1. head not boxed in — every live head has ≥ 1 move
2. reachability — each colour's head can still reach its endpoint
3. coverage — every empty node reachable by **some** live head
4. no stranded pocket — an empty region with no head in it = dead
5. deadend — an empty node with < 2 usable edges can't be a degree-2 pass-through

All 5 walk **NODES** and **EDGES**. That's why they handle every variant for free:

- ↳ a wall is just an absent edge → 2/3/5 route around it automatically
- ↳ a hole is just an absent node → never enters #3's count
- ↳ a seam is just an extra edge → #2/#3 traverse it, no special case
- ↳ ZERO lines of variant-specific pruning code.

↳ human heuristics I encoded: corners & edges first · forced moves · don't strand a region ·
 don't hug yourself · most-constrained first

o) THE REAL GOAL

a bubble floating over the game. Tap it → it reads the board and solves it —

↳ and shows me NOTHING. Board stays clean. Then it **asks**.

tap → capture → read + solve (silent) → **ASK ME**

⚡ DRAW IT

↳ app swipes each flow itself, one by one

↳

`AccessibilityService.dispatchGesture()`

↳ real touch events. game can't tell.

no root.

👋 MANUAL

↳ **mild** overlay — ~35% opacity

↳ + direction arrows (a line alone doesn't say which end to start from!)

↳ I trace it myself.

↳ why not show it instantly? because half the time drawing it myself is the whole point.

Keep the choice.

o) Pipeline

Foreground service → bubble (**SYSTEM_ALERT_WINDOW**) → `MediaProjection` 1 frame → grid detection → solver → overlay / gestures

⚠ THE CONSTRAINT THAT KILLS IT

overlay MUST be **FLAG_NOT_TOUCHABLE**.

↳ else it swallows my finger and I can't trace the answer it's showing me.

↳ looks perfect in a screenshot. unusable in the hand. Only the bubble takes touches.

o) WHERE THIS BREAKS

1. Grid detection is the hard part — not the solver.

↳ themes, DPI, notches, ad banners. And telling a wall from a hole from a gridline off a screenshot.

↳ a wall is a line **on a border**; a hole is the **cell itself**. Discriminate on thickness + brightness, not presence.

↳ **one missed wall = board unsolvable, with no clue why.**

2. Auto-draw is DESTRUCTIVE when wrong.

↳ a few px of calibration error → swipe starts on the wrong cell → drags a flow I didn't mean and wrecks a half-finished board.

↳ so: gate it on a confident read. The parity check (p.9) is a free gate.

3. FLAG_SECURE — if the game ever sets it, MediaProjection returns a black frame.

↳ no workaround. Not true today. One release away.

4. Speed — dispatchGesture too fast → game drops cells mid-drag.

↳ needs a real knob. ~40-60ms per cell. Interpolate **per cell**, not just at corners.

o) STILL UNKNOWN — must check a real screenshot

? Links (Hoop/Loop/Chain) — board shapes, or **portals**? Either way it's a parse-time transform, so not blocking.

? Cubes — exact net layout & seam orientation.

NEXT →

1. finish solver core + text format (WALL / HOLE / BRIDGE) + CLI
2. variant fixtures — the **load-bearing** ones (drop a wall → must go unsolvable)
3. grid detection from screenshots
4. Android shell + Manual mode
5. Auto-draw